

Santo Pegasus Soluciones: Guía Oficial de Ingeniería Back-end

Versión del Documento: 2.4.0

Última Actualización: Octubre de 2025

Departamento: Ingeniería de Software / Chapter de Back-end

1. Introducción

Bienvenido a la Guía de Ingeniería Back-end de **Santo Pegasus Soluciones**. Este documento sirve como la "fuente de la verdad" para todos los desarrolladores de nuestro equipo. Nuestro objetivo es garantizar que el código que producimos sea escalable, sostenible, legible y seguro.

La alineación con estas directrices es obligatoria para todos los nuevos proyectos y fuertemente recomendada para refactorizaciones de sistemas heredados. La conformidad con esta guía asegura que Santo Pegasus mantenga un alto estándar de entrega técnica, facilitando la colaboración entre diferentes equipos y la evolución continua de nuestra infraestructura de software.

2. Principios Fundamentales de Ingeniería

En Santo Pegasus, basamos nuestras decisiones arquitectónicas y de codificación en principios universales de ingeniería de software. Todos los desarrolladores deben tener los siguientes conceptos internalizados para garantizar que el código producido sea escalable, sostenible, legible y seguro:

2.1. SOLID

Los cinco principios de la programación orientada a objetos que facilitan el mantenimiento y la expansión del código:

- **S - Single Responsibility Principle (SRP):** Una clase debe tener uno, y solo un, motivo para cambiar. Este principio fundamental dicta que la cohesión debe ser alta, garantizando que cada componente del sistema ejecute una única tarea lógica. En Santo Pegasus, es obligatorio separar las responsabilidades en capas distintas: por ejemplo, la lógica de persistencia y comunicación con la base de datos debe residir exclusivamente en los **Repositories**, jamás insertándose directamente en los **Controllers**, que deben enfocarse solo en el manejo de peticiones y respuestas HTTP. Al aplicar el SRP, reducimos el acoplamiento, facilitamos la creación de pruebas unitarias y minimizamos el

riesgo de efectos secundarios durante mantenimientos futuros.

- **O - Open/Closed Principle (OCP):** Las entidades de software deben estar abiertas para extensión, pero cerradas para modificación. En Santo Pegasus, priorizamos el uso de interfaces y abstracciones para permitir que se agreguen nuevos comportamientos sin necesidad de alterar el código fuente original ya consolidado. Esto es crucial para evitar la introducción de regresiones en sistemas estables y facilitar la implementación de nuevos algoritmos de procesamiento, como ocurre en nuestros servicios de pago.
- **L - Liskov Substitution Principle (LSP):** Las clases derivadas deben ser capaces de sustituir a sus clases base sin comprometer la integridad de la aplicación. En Santo Pegasus, esto significa que una subclase nunca debe eliminar el comportamiento de la superclase o lanzar excepciones inesperadas para métodos heredados, garantizando que el polimorfismo funcione de forma predecible en todos nuestros servicios.
- **I - Interface Segregation Principle (ISP):** Es preferible tener múltiples interfaces específicas a una única interfaz genérica. No se debe obligar a una clase a implementar métodos que no utilizará. En Santo Pegasus, aplicamos este principio para evitar la creación de "interfaces gordas" que contaminan el código y aumentan el acoplamiento innecesario. Al segmentar interfaces, garantizamos que las clases clientes dependan solo de los métodos que realmente necesitan, facilitando el mantenimiento y la evolución modular de nuestros servicios back-end.
- **D - Dependency Inversion Principle (DIP):** Dependencia de abstracciones y no de implementaciones concretas. En Santo Pegasus, este principio es la base para un sistema verdaderamente desacoplado. En lugar de que una clase de alto nivel instancie directamente una clase de bajo nivel, ambas deben depender de interfaces. Utilizamos extensivamente la **Inyección de Dependencias de Spring** para gestionar el ciclo de vida de los objetos y viabilizar este principio, permitiendo que las implementaciones reales sean intercambiadas sin afectar el flujo principal del negocio. Esto facilita el mantenimiento, permite la sustitución de proveedores externos y hace que las pruebas unitarias con *mocks* sean mucho más simples y eficaces.

2.2. DRY, KISS y YAGNI

Además de SOLID, aplicamos los principios DRY (Don't Repeat Yourself) para evitar la duplicación de lógica, KISS (Keep It Simple, Stupid) para mantener un diseño simple y legible, y YAGNI (You Aren't Gonna Need It) para evitar la implementación de funcionalidades innecesarias para el momento actual. En **Santo Pegasus Soluciones**, la aplicación de **DRY** significa que, al identificar la misma lógica en múltiples lugares, esta debe extraerse a métodos utilitarios o servicios base, garantizando que una alteración en una regla de negocio deba hacerse en un solo lugar.

El principio **KISS** es nuestro norte para el diseño de sistemas; priorizamos la legibilidad y la facilidad de mantenimiento en detrimento de soluciones excesivamente ingeniosas o complejas que puedan oscurecer el funcionamiento del código para otros miembros del equipo.

Finalmente, **YAGNI** nos protege contra el desperdicio de esfuerzo de ingeniería, orientando al

equipo a implementar estrictamente lo necesario para los requisitos actuales, sin agregar funcionalidades basadas en suposiciones de necesidades futuras que pueden nunca concretarse.

2.2.1. DRY (Don't Repeat Yourself)

Evite la duplicación de código. Si encuentra la misma lógica escrita en dos lugares diferentes, extráigala a un método utilitario, servicio o clase base. La aplicación de este principio garantiza que una alteración en una regla de negocio se realice en un solo lugar, reduciendo drásticamente el riesgo de inconsistencias en el sistema. Además de la extracción a métodos, recomendamos el uso de constantes para valores fijos y la centralización de validaciones comunes en clases utilitarias o en los propios DTOs utilizando Bean Validation, evitando que la misma validación se repita en múltiples Services.

2.2.2. KISS (Keep It Simple, Stupid)

La simplicidad es el objetivo principal del diseño en **Santo Pegasus Soluciones**. Este principio sirve como nuestro norte para el desarrollo de sistemas, donde priorizamos la legibilidad y la facilidad de mantenimiento sobre soluciones excesivamente ingeniosas o complejas que puedan oscurecer el funcionamiento del código para otros miembros del equipo.

Evitar la complejidad innecesaria significa que un código legible y simple de entender es siempre preferible a un código "inteligente", pero oscuro y complejo. En la práctica, esto se traduce en evitar abstracciones prematuras y garantizar que la lógica del negocio se exprese de forma clara y directa, facilitando la evolución continua de nuestra infraestructura de software y la colaboración entre diferentes equipos.

2.2.3. YAGNI (You Aren't Gonna Need It)

El principio **YAGNI** actúa como una salvaguarda contra el desperdicio de esfuerzo de ingeniería en **Santo Pegasus Soluciones**. Orienta al equipo a no agregar funcionalidades basadas en suposiciones de necesidades futuras que pueden nunca concretarse.

En la práctica, la aplicación de este principio implica:

- **Enfoque en el requisito actual:** Implemente estrictamente lo que es necesario para atender el requisito de negocio presente.
- **Reducción de complejidad:** Al evitar el desarrollo prematuro de recursos "útiles en el futuro", mantenemos la base de código más limpia, facilitando el mantenimiento y la comprensión por parte de otros desarrolladores.
- **Eficiencia de recursos:** Protegemos al equipo contra el gasto de tiempo en códigos que pueden necesitar ser eliminados o reescritos totalmente en caso de que la necesidad futura cambie de dirección.

Este principio es un complemento esencial al diseño simple (KISS), garantizando que la

evolución de nuestra infraestructura esté guiada por necesidades reales e inmediatas.

3. Patrones Arquitectónicos: Java y Spring Boot

Nuestro ecosistema principal está basado en **Java 17+** y **Spring Boot 3+**. Para mantener la consistencia, adoptamos una arquitectura en capas (Layered Architecture):

3.1. División de Capas

- **Controllers (@RestController):** Responsables exclusivamente de recibir peticiones HTTP, validar los parámetros de entrada (usando Bean Validation), llamar al Service adecuado y retornar la respuesta HTTP correcta (con los debidos Status Codes).
Ninguna regla de negocio debe residir aquí.
- **Services (@Service):** El corazón de la aplicación. Aquí reside toda la regla de negocio. El Service orquesta las llamadas a los repositorios y servicios externos.
- **Repositories (@Repository):** Interfaces que extienden JpaRepository o MongoRepository. Responsables de la comunicación exclusiva con la base de datos.

3.2. Transferencia de Datos (DTOs y Mappers)

Nunca retorne entidades de base de datos (ej: clases anotadas con @Entity) directamente en los Controllers.

- Use el Patrón **DTO (Data Transfer Object)** para la comunicación entre la API y los clientes.
- Utilice bibliotecas como **MapStruct** para convertir Entidades en DTOs y viceversa, evitando código *boilerplate* de mapeo manual.

3.3. Manejo Global de Excepciones

Todas las APIs de Santo Pegasus deben implementar un @ControllerAdvice o @RestControllerAdvice.

- Las excepciones de negocio deben ser personalizadas (ej: ResourceNotFoundException, BusinessException).
- El retorno de error debe seguir la especificación RFC 7807 (Problem Details for HTTP APIs), estandarizando la estructura JSON de errores devuelta al *front-end*.

3.4. Documentación de APIs

Para garantizar la interoperabilidad y facilitar la integración con aplicaciones front-end y clientes externos, todas las APIs deben estar rigurosamente documentadas utilizando la especificación OpenAPI (Swagger). La documentación debe generarse automáticamente a partir del código (utilizando bibliotecas como springdoc-openapi) y exponer claramente los *endpoints*,

parámetros esperados, esquemas de *payload* y códigos de respuesta HTTP de forma actualizada.

3.5. Comunicación Síncrona entre Microservicios

En escenarios donde la comunicación síncrona entre servicios internos es estrictamente necesaria (evitando el acoplamiento temporal siempre que sea posible), estandarizamos la utilización del WebClient del ecosistema Spring WebFlux para llamadas REST. Para rutas críticas que demandan altísimo rendimiento y baja latencia, se fomenta fuertemente el uso de gRPC. La utilización de clientes heredados, como RestTemplate, está descontinuada para nuevos proyectos.

4. Patrones de Diseño (Design Patterns)

Fomentamos el uso de Design Patterns consolidados (GoF) para resolver problemas recurrentes y garantizar una base de código flexible:

1. **Strategy:** Este patrón es esencial en nuestros flujos de pago. Permite alternar algoritmos de procesamiento (como Pix, Tarjeta o Boleto) dinámicamente en tiempo de ejecución. Al encapsular cada método de pago en su propia clase de estrategia, eliminamos estructuras condicionales (if/else o switch/case) extensas y complejas en el servicio principal, facilitando la adición de nuevos métodos sin impactar el código existente. En términos prácticos, el sistema se vuelve más modular, ya que la lógica específica de cada proveedor o medio de pago queda aislada, permitiendo que el *Service* solo orqueste la ejecución a través de una interfaz común.
2. **Builder:** Este patrón es un requisito obligatorio para la instanciación de objetos complejos, garantizando la inmutabilidad de las instancias y la legibilidad del código al evitar el uso de constructores con largas listas de parámetros. Priorizamos la utilización de la anotación `@Builder` de la biblioteca **Lombok**, lo que reduce drásticamente el código *boilerplate*. Su aplicación es fundamental en escenarios de pruebas unitarias, permitiendo la creación de escenarios de prueba específicos con claridad y agilidad. Además, el uso de builders facilita la evolución de las clases, ya que la adición de nuevos campos no rompe las instancias existentes en otras partes del sistema, manteniendo la integridad y facilitando mantenimientos futuros.
3. **Factory Method:** Este patrón de diseño creacional se aplica cuando la creación de un objeto exige una lógica decisoria compleja o validaciones previas que no deben sobrecargar a la clase cliente. Al centralizar la responsabilidad de instanciación en un método o clase específica, promovemos el desacoplamiento entre el cliente y las implementaciones concretas, facilitando la introducción de nuevos tipos de objetos sin la necesidad de modificar el código que los utiliza.

- **Aplicaciones Prácticas:** Utilizamos el *Factory Method* para instanciar componentes que dependen de configuraciones dinámicas o parámetros específicos del entorno, garantizando que el objeto resultante esté siempre en un estado válido y listo para uso.
 - **Beneficios para el Equipo:** La adopción de este patrón reduce la complejidad en los **Services**, pues elimina la necesidad de múltiples bloques *if/else* o *switch* destinados solo a la creación de instancias, haciendo el flujo principal de negocio mucho más legible y fácil de mantener.
4. **Observer:** Este patrón de comportamiento es fundamental para mantener el desacoplamiento en nuestra arquitectura orientada a eventos. En Santo Pegasus, se implementa vía *ApplicationEventPublisher* de Spring para gestionar comunicaciones asíncronas de forma eficiente.
- **Casos de Uso:** Se utiliza principalmente para envíos de correos electrónicos post-registro, integración con servicios de notificación (como el microservicio que utiliza AWS SES y SQS) y actualizaciones de registros de auditoría, sin que el *Service* de negocio necesite conocer los detalles de estas implementaciones.
 - **Ventajas:** Al adoptar el *Observer*, garantizamos que los servicios principales permanezcan enfocados exclusivamente en sus reglas de negocio fundamentales, reduciendo la complejidad cognitiva del código y facilitando la inclusión de nuevos oyentes (*listeners*) para un mismo evento sin alterar el código del publicador.

5. Implementaciones de IA y Uso de RAG

Con la llegada de las IAs Generativas, Santo Pegasus integró componentes inteligentes en sus back-ends. Para casos donde el LLM necesita consultar nuestros datos internos, utilizamos la arquitectura **RAG (Retrieval-Augmented Generation)**. Este enfoque permite que el modelo genere respuestas basadas en hechos y conocimientos específicos de la empresa, que no formaban parte de su entrenamiento original.

El funcionamiento del RAG en Santo Pegasus sigue cuatro etapas principales: primero, la pregunta del usuario se usa como query para buscar fragmentos semánticamente relevantes en una base de datos externa (Retrieval); luego, esos fragmentos son extraídos (Context Extraction); después, el contexto recuperado se combina con la pregunta original para formar un prompt robusto; y, finalmente, el LLM genera la respuesta fundamentada (Generation). Esta estrategia es vital para automatizar atenciones operativas, como dudas frecuentes de RRHH sobre beneficios y políticas internas, liberando a los equipos para funciones estratégicas.

5.1. Reglas para Pipelines RAG

- **Chunking (División de Texto):** Textos largos deben ser divididos en fragmentos (chunks) de máximo **512 tokens**, con un *overlap* (superposición) de 50 tokens para

mantener el contexto entre los párrafos.

- **Embeddings:** El modelo estándar homologado para generar vectores en Santo Pegasus es el text-embedding-004.
- **Base de Datos Vectorial:** Utilizamos **Pinecone** o **Qdrant** en producción. Las bases vectoriales en memoria (como FAISS) están permitidas solo para pruebas locales.
- **Recuperación (Retrieval):** El número máximo de documentos recuperados (K) para componer el contexto del *prompt* del LLM debe ser **K=4**. Un número mayor causa degradación del rendimiento y alucinaciones.
- **Orquestación:** Todo el flujo de RAG debe ser orquestado vía **LangChain** o **LlamaIndex** en la capa de Service.

6. Control de Versiones y GitFlow

6.1. Patrón de Nomenclatura de Ramas (Branches)

Utilizamos una variación simplificada de GitFlow. El nombre de las ramas debe seguir el patrón:

- feature/TICKET-123-nombre-de-la-feature
- bugfix/TICKET-456-descripcion-del-bug
- hotfix/TICKET-789-correccion-critica-prod

6.2. Commits Semánticos (Conventional Commits)

Los mensajes de commit deben obligatoriamente seguir la especificación de *Conventional Commits*:

- feat: agrega endpoint de pago PIX
- fix: corrige cálculo de impuesto en la factura
- refactor: mejora rendimiento del mapper de usuarios
- docs: actualiza readme con instrucciones de setup local
- test: agrega pruebas unitarias para el UserService

7. Proceso de Code Review

El Code Review es el pilar de nuestra calidad de software. Ningún código va a la rama main sin pasar por ese proceso.

7.1. Reglas para el Autor (Quien abre el Pull Request)

1. Garantice que el código compila y que la cobertura de pruebas unitarias sea **igual o superior al 80%** (medido vía JaCoCo/SonarQube).
2. El PR no debe contener más de 400 líneas de código alteradas. Si es mayor, divídalo en

PRs más pequeños.

3. Complete el *template* del PR explicando "Qué se hizo", "Por qué se hizo" y "Cómo probar".

7.2. Reglas para el Revisor

1. **Aprobaciones:** Todo Pull Request (PR) en Santo Pegasus exige **al menos 2 aprobaciones** de miembros Senior o Semi-Senior (Pleno) antes del merge. Este criterio garantiza que el código sea revisado desde diferentes perspectivas, reduciendo fallas técnicas y diseminando el conocimiento sobre las alteraciones entre el equipo.
2. **Tono y Etiqueta:** La comunicación debe estar siempre pautada por la empatía y la educación. El enfoque de la revisión debe ser exclusivamente la mejora del código, nunca una crítica al autor. Se recomienda el uso de preguntas sugestivas para incentivar la reflexión del desarrollador, como: "*¿Qué te parece extraer esto a un método privado?*", evitando imperativos como "*Cambia esto a un método privado*".
3. **Criterios de Revisión:** El revisor actúa como un guardián de la base de código, debiendo validar minuciosamente la legibilidad (facilidad de comprensión), la seguridad (prevención de vulnerabilidades), el rendimiento (uso eficiente de recursos) y, fundamentalmente, si la implementación respeta rigurosamente todas las directrices establecidas en la Guía de Ingeniería Back-end.

8. Seguridad y Buenas Prácticas Generales

8.1. Autenticación y Autorización

El control de acceso a nuestras APIs debe gestionarse de forma centralizada utilizando Spring Security. Adoptamos el uso de tokens JWT (JSON Web Tokens) basados en el flujo del protocolo OAuth 2.0 u OpenID Connect (OIDC). La extracción y validación de permisos (*roles*) deben ocurrir a nivel de *endpoint*, garantizando que solo los clientes autenticados y con los privilegios correctos ejecuten operaciones sensibles.

8.2. Credenciales y Secrets

NUNCA haga *hardcode* (escribir directamente en el código) de contraseñas, claves de API, URLs de base de datos o tokens de servicios de IA. La exposición de credenciales en repositorios de código es una de las vulnerabilidades más críticas y puede llevar al compromiso total de la infraestructura. Para garantizar la protección de estos datos sensibles, siga rigurosamente las directrices a continuación:

- **Uso de Bóvedas de Contraseñas:** En entornos de producción y homologación, es obligatorio el uso de soluciones de gestión de secretos, como **HashiCorp Vault**, **AWS Secrets Manager** o **Google Cloud Secret Manager**. Integre estos servicios a Spring

Boot utilizando *Spring Cloud Config* para que los secretos sean inyectados en tiempo de ejecución sin tocar nunca el disco o el repositorio Git.

- **Sanitización de Repositorios:** Utilice herramientas de escaneo automático, como *gitleaks* o *trufflehog*, en nuestros pipelines de CI/CD para detectar y bloquear commits que contengan cadenas de texto que se asemejen a secretos (claves privadas, tokens JWT, contraseñas).
- **Segregación de Acceso:** Las credenciales deben ser exclusivas por entorno. Jamás utilice una contraseña de base de datos de producción en un entorno de desarrollo. El principio de privilegio mínimo debe ser aplicado, garantizando que cada microservicio tenga acceso solo a los secretos estrictamente necesarios para su operación.
- **Rotación de Credenciales:** Establezca un proceso de rotación periódica para claves de API y tokens de servicios externos, minimizando el impacto en caso de que ocurra una filtración accidental.

8.3. Variables de Entorno

Para garantizar la seguridad y la flexibilidad de las configuraciones en diferentes entornos (desarrollo, homologación y producción), utilice siempre **variables de entorno** para gestionar datos sensibles y parámetros volátiles. En el ecosistema Java y Spring Boot, la recuperación de estos valores debe hacerse preferiblemente a través de:

- **Inyección con @Value:** Utilice la anotación `@Value("${NOMBRE_DE_LA_VARIABLE}")` para inyectar valores directamente en atributos de clases gestionadas por Spring.
- **Acceso Nativo:** Use `System.getenv()` cuando sea necesario acceder a variables fuera del contexto de inyección de Spring.

Durante el **desarrollo local**, está permitida y fomentada la utilización de archivos **.env** para emular el entorno de ejecución. Sin embargo, por cuestiones críticas de seguridad, estos archivos **deben estar obligatoriamente listados en el .gitignore** para evitar la filtración de credenciales en el repositorio Git.

8.4. Logging

La utilización de `System.out.println()` para fines de depuración o rastreo en entornos profesionales está estrictamente prohibida en **Santo Pegasus**. Esta práctica es ineficiente, ya que no permite la categorización por niveles de severidad, dificulta el filtrado en herramientas de agregación de logs y causa un impacto negativo en el rendimiento de la aplicación debido al comportamiento bloqueante del flujo de salida estándar. Estandarizamos el uso de **SLF4J** (Simple Logging Facade for Java) como interfaz de abstracción, teniendo a **Logback** como framework de implementación nativo de Spring Boot. Las siguientes directrices deben seguirse rigurosamente para la gestión de logs:

- **Niveles de Log en Producción:** El nivel predeterminado debe configurarse como **INFO**. El nivel **DEBUG** solo debe habilitarse temporalmente para la solución de problemas

específicos, buscando evitar el almacenamiento innecesario de grandes volúmenes de datos.

- **Manejo de Errores:** Las fallas inesperadas o excepciones de sistema deben ser registradas obligatoriamente en el nivel **ERROR**. Es fundamental que el mensaje de error venga acompañado de la *stacktrace* completa para viabilizar el análisis de causa raíz por parte de ingeniería.
- **Privacidad y Seguridad:** Está terminantemente prohibido registrar información sensible (PII) en los logs, tales como contraseñas, claves de API, números de CPF o datos de pago. En el proyecto *Agendio*, por ejemplo, el ID de la consulta puede registrarse para rastreo, pero los datos personales de los pacientes deben omitirse.
- **Logs Estructurados:** Siempre que sea posible, utilice logs en formato JSON para facilitar la ingesta por sistemas como ELK Stack o Datadog.

8.5. Validación de Entradas

La integridad de los datos y la seguridad de la aplicación dependen fundamentalmente de la premisa de que el back-end es la última y más importante línea de defensa. En Santo Pegasus, seguimos la regla de **nunca confiar en los datos enviados por el cliente**, independientemente de cuán robusta sea la validación en el front-end. Implementamos validaciones estrictas en múltiples capas para mitigar riesgos de *SQL Injection*, *Cross-Site Scripting* (XSS) y la persistencia de datos corruptos que puedan comprometer la lógica de negocio. Para garantizar la conformidad técnica, las siguientes prácticas son obligatorias:

- **Bean Validation (JSR 380):** Utilice anotaciones como `@NotNull`, `@Size`, `@Email` y `@Pattern` directamente en los DTOs de entrada de los Controllers.
- **Sanitización de Cadenas de Texto (Strings):** Elimine caracteres especiales o scripts maliciosos de entradas de texto que se mostrarán posteriormente a otros usuarios.
- **Validación de Tipos y Formatos:** Asegúrese de que los IDs, fechas y valores monetarios sigan estrictamente el formato esperado por la aplicación antes de cualquier procesamiento por capas de servicio.

8.6. Observabilidad Integral: Métricas y Tracing

Además del uso ya establecido de SLF4J/Logback para logs estructurados, nuestros servicios deben ser plenamente observables para garantizar la resiliencia operativa. La observabilidad en **Santo Pegasus** se basa en tres pilares fundamentales: logs, métricas y rastreo.

- **Métricas de Rendimiento y Salud:** Es obligatoria la exposición de métricas detalladas utilizando **Spring Boot Actuator** y **Micrometer**. Se debe configurar la exportación nativa hacia **Prometheus**, permitiendo el monitoreo en tiempo real de latencia, tasa de errores, consumo de CPU y uso de memoria.
- **Rastreo Distribuido (Distributed Tracing):** En nuestra arquitectura de microservicios, es obligatorio implementar soluciones de rastreo para propagar el *Trace ID* y el *Span ID* entre las peticiones. Esto facilita la depuración de flujos complejos y la identificación de

cuellos de botella en sistemas distribuidos.

- **Health Checks:** Los endpoints de *health* deben reflejar el estado real de las dependencias críticas (bases de datos, brokers de mensajes), permitiendo que el orquestador de contenedores tome decisiones precisas sobre el ciclo de vida de las instancias.

La implementación rigurosa de estas prácticas asegura que el equipo de ingeniería pueda responder proactivamente a incidentes y mantener el alto estándar de disponibilidad exigido por Santo Pegasus.

9. Gestión de Base de Datos y Migrations

La evolución del esquema de bases de datos relacionales en **Santo Pegasus** debe ser estrictamente controlada, versionada y automatizada. Está terminantemente prohibida la alteración manual de estructuras de datos, como la creación de tablas o modificación de columnas, así como la ejecución de scripts DDL directamente en entornos de producción por medio de clientes SQL. Para garantizar que el estado de la base de datos evolucione de forma predecible, auditable y sincronizada con la versión de la aplicación, adoptamos las siguientes directrices:

- **Herramientas de Migration:** Utilizamos obligatoriamente herramientas como **Flyway** o **Liquibase**. Estas herramientas permiten que cada alteración en la base de datos sea tratada como un artefacto de software, versionado en Git junto al código fuente.
- **Implantabilidad y Rollback:** Cada script de migración debe ser idempotente y probado exhaustivamente en entornos de homologación. Se recomienda el uso de **Flyway** por su simplicidad al gestionar el historial a través de una tabla de metadatos, garantizando la consistencia del esquema en todos los nodos del clúster.
- **Compatibilidad con Arquitectura Hexagonal:** Siguiendo nuestros patrones de desacoplamiento, la lógica de migración debe residir en la capa de infraestructura, garantizando que el núcleo del negocio no dependa de detalles de implementación de la base de datos (como el motor **PostgreSQL**).
- **Entornos Gestionados:** En infraestructuras en la nube (cloud), como **AWS RDS** u **Oracle Autonomous Database**, el control a través de migrations es vital para mantener la paridad entre entornos de desarrollo local y producción, facilitando la automatización vía pipelines de CI/CD.

Al mantener un registro histórico de todas las alteraciones, aseguramos la trazabilidad necesaria para auditorías y la resiliencia operativa indispensable para nuestros sistemas de misión crítica.

10. Estrategia y Cobertura de Pruebas

La calidad del código debe ser asegurada por un enfoque multifacético. Aunque la cobertura mínima del 80% en pruebas unitarias es obligatoria en la etapa de *Code Review*, nuestra ingeniería exige el mantenimiento de una pirámide de pruebas equilibrada:

- **Pruebas Unitarias:** Enfoque en la lógica de negocio aislada en *Services* y utilitarios, utilizando *mocks* para dependencias externas vía Mockito.
- **Pruebas de Integración:** Obligatorias para funcionalidades críticas y comunicaciones con bases de datos. Recomendamos el uso de **Testcontainers** para garantizar el aislamiento y la paridad con el entorno productivo.
- **Pruebas de Contrato:** Implementadas en APIs vitales para prevenir rupturas de compatibilidad inesperadas con sistemas consumidores y garantizar la estabilidad del ecosistema de microservicios.

11. CI/CD y Proceso de Deploy

El ciclo de vida de nuestras aplicaciones es automatizado a través de pipelines de Integración y Entrega Continuas (CI/CD). Tras la aprobación y el *merge* de un *Pull Request*, el proceso sigue directrices rigurosas:

- **Contenedorización:** El uso de **Docker** es obligatorio para asegurar la inmutabilidad del artefacto, garantizando que el mismo código validado en *Staging* (homologación) sea promovido a *Producción* sin variaciones.
- **Estrategias de Disponibilidad:** Para evitar cualquier indisponibilidad durante las actualizaciones, utilizamos orquestadores que soportan **Blue-Green** o **Canary Releases**, permitiendo reversiones rápidas en caso de anomalías.
- **Automatización de Infraestructura:** En entornos como AWS RDS, las alteraciones de la base de datos se integran vía herramientas de *migration* (Flyway/Liquibase) dentro del propio pipeline.